DOCKER · COHORT NOTES

Docker Compose Dev Environment Setup

How to run your full-stack app in containers during development — with hot reload, no rebuilds on every change, and zero "works on my machine" drama.

01 What we're solving

Docker normally works like: build image → run container. That's great for production. But in dev you change code every 5 minutes. Rebuilding the image each time is insane — it takes 30–90 seconds and breaks your flow entirely.

The dev setup solves this with two things working together:

1. **Bind mounts** — your local source code is mounted directly into the container. Edit a file locally, the container sees it instantly.
2. **Named volumes for node_modules** — the container keeps its own isolated dependencies, not your host machine's.

> **MENTAL MODEL**
> Build the image once to set up the OS and install dependencies. After that, your code flows into the container live via a bind mount. `nodemon` or Vite handles the rest.

02 Project structure

project root — tree

```tree
my-app/
├── docker-compose.yml
├── server/
│   ├── Dockerfile
│   ├── package.json
│   └── src/
│       └── index.ts
└── client/
    ├── Dockerfile
    ├── package.json
    └── src/
        └── main.tsx
```

Each service (backend, frontend) has its own `Dockerfile`. The `docker-compose.yml` at the root wires everything together.

03 Dockerfiles for dev

Dev Dockerfiles are intentionally minimal. Their only job is: set up Node, install deps. Your actual code comes in via the bind mount — it doesn't need to be `COPY`'d into the image.

server/Dockerfile — dockerfile

```dockerfile
FROM node:20-alpine

WORKDIR /app

# Copy only package files first — this layer gets cached
COPY package*.json ./

# Install deps inside the container (Linux binaries)
RUN npm install

# Source code comes in via bind mount — no COPY needed in dev

CMD ["npm", "run", "dev"]
```

> **LAYER CACHING RULE**
> Always `COPY package*.json ./` before `RUN npm install`. If you change a `.ts` file, Docker skips the npm install layer entirely — it only re-runs it when `package.json` actually changes.

04 The compose file — full breakdown

docker-compose.yml — yaml

```yaml
services:

  backend:
    build: ./server                    # build image from server/Dockerfile
    ports:
      - "5000:5000"                    # HOST:CONTAINER
    volumes:
      - ./server:/app                  # bind mount: your code → container
      - backend-node-modules:/app/node_modules  # named volume: protect deps
    environment:
      - MONGO_URI=your_atlas_uri
      - PORT=5000
    command: npm run dev               # overrides Dockerfile CMD (nodemon)

  frontend:
    build: ./client
    ports:
      - "3000:5173"                    # host 3000 → vite's 5173
    volumes:
      - ./client:/app
      - frontend-node-modules:/app/node_modules
    command: npm run dev -- --host     # --host exposes vite outside container

# All named volumes must be declared here
volumes:
  backend-node-modules:
  frontend-node-modules:
```

Now let's break down every keyword so nothing is hand-waved.

`services:`
Defines the containers in your system. Each key under this is a service — backend, frontend, mongo, redis, etc.

`build:`
Path to the folder containing the Dockerfile for that service. Docker reads it and builds the image.

`ports: HOST:CONTAINER`
Maps a port on your machine to a port inside the container. Without this, the container runs but you can't reach it.

`volumes:`
The most important part. Controls what storage is shared between your machine and the container — and what stays isolated.

`environment:`
Injects env variables into the container. Your Express app reads `process.env.MONGO_URI` from here.

`command:`
Overrides the `CMD` in your Dockerfile. Used to run `nodemon` or Vite's dev server with the right flags.

05 Volumes — the part everyone gets wrong

Stop thinking of a volume as "a folder." A volume is **controlled leakage of Docker's isolation**. You decide what flows in, what flows out, and what stays fully isolated.

WITHOUT PROPER VOLUMES

- ✗ Code changes require a full image rebuild
- ✗ Bind mount overwrites node_modules in the container
- ✗ macOS/Windows node_modules used on Linux → binary mismatch
- ✗ Rollup / esbuild native bindings break

WITH THE CORRECT SETUP

- ✓ Code changes reflect instantly — no rebuild
- ✓ Container has its own isolated node_modules
- ✓ Native binaries compiled for Linux inside the container
- ✓ Both frontend and backend reload automatically

volume types explained — yaml

```yaml
volumes:
  # 1. Bind Mount — ties a host folder to a container path
  - ./server:/app              # changes on host instantly visible in container

  # 2. Named Volume — Docker manages the storage, not you
  - backend-node-modules:/app/node_modules   # survives restarts, stays isolated

  # 3. Anonymous Volume (old pattern, avoid this)
  - /app/node_modules          # works but can't be reused or inspected easily
```

> **WHY NAMED VOLUME BEATS ANONYMOUS VOLUME FOR NODE_MODULES**
> Anonymous volumes (`- /app/node_modules`) work, but named volumes are explicit — you can inspect them with `docker volume inspect`, they're clearly declared at the bottom of the file, and you can wipe them precisely with `docker volume rm` when debugging.

Here's the mental model for where everything actually lives:

storage map — text

```text
Your Laptop (Host)                    Container (/app)
─────────────────                     ────────────────
server/src/index.ts   ←— bind —→      /app/src/index.ts
server/package.json   ←— bind —→      /app/package.json
                                      /app/node_modules  ← named volume (Docker owns this)

Changes you make on host ————————→ container sees them INSTANTLY
node_modules on host ————————————→ IGNORED (named volume takes priority)
```

06 Adding new npm packages

This is where most people get confused. You have two options.

Option 1 — install inside the running container (recommended for day-to-day):

```
$ docker exec -it <container_name> npm install axios
# package.json updates on your host (bind mount), node_modules updated inside container
```

Option 2 — install locally, then rebuild (for clean state):

```
$ npm install axios
$ docker compose up --build # rebuilds image, re-runs npm install inside
```

> **IMPORTANT**
> When you use Option 1, `package.json` updates because it's a bind-mounted file. Commit it to git as usual. Anyone who clones the repo and runs `docker compose up --build` will get the correct deps.

07 Commands you'll actually use

```
$ docker compose up --build
# build images + start all containers. Use this first time or after package changes.
```

```
$ docker compose up
# start containers using existing images (no rebuild). Use for daily dev.
```

```
$ docker compose down
# stop and remove containers. Named volumes are preserved.
```

```
$ docker compose down -v
# stop containers AND wipe all named volumes. Nuclear option for debugging.
```

```
$ docker compose logs -f backend
# tail logs for a specific service
```

```
$ docker exec -it <container> sh
# open a shell inside a running container
```

08 When do you actually need to rebuild?

SCENARIO	ACTION NEEDED	REASON
Changed .ts / .js file	Nothing	Bind mount + nodemon handles it
Changed .env file	Restart only	`docker compose restart backend`
Added npm package	exec or --build	node_modules needs updating in container
Changed Dockerfile	--build	Image layer changed
Upgraded base image (node:20 → 22)	--build	New base image must be pulled and built

09 Common mistakes

> **MISTAKE 1 — USING LOCALHOST INSIDE A CONTAINER**
> Containers are isolated. `localhost` inside a container refers to that container itself, not your host or another container. Use the service name instead: `mongodb://mongo:27017` not `mongodb://localhost:27017`.

> **MISTAKE 2 — NOT PROTECTING NODE_MODULES WITH A NAMED VOLUME**
> If you only have `./server:/app` and no named volume for `node_modules`, the bind mount will overwrite the container's deps with whatever (or nothing) is on your host. Always add `backend-node-modules:/app/node_modules`.

> **MISTAKE 3 — FORGETTING --HOST FOR VITE**
> Vite by default binds to `127.0.0.1` inside the container, which is not reachable from outside. You must run `vite --host` (or add `host: true` in `vite.config.ts`) to expose it on `0.0.0.0` so port mapping works.

> **MISTAKE 4 — NOT CLEANING UP WHEN THINGS BREAK**
> Docker caches aggressively. When something mysteriously breaks, run `docker compose down -v` to wipe containers and volumes, then `docker compose up --build` for a clean start. This fixes 80% of "it worked yesterday" situations.

10 Final takeaway

> **THE ONE THING TO REMEMBER**
> Docker is isolation. Volumes are controlled leaks of that isolation. Your job as a developer is to decide what should leak (source code → bind mount) and what shouldn't (dependencies → named volume). Get that separation right and the rest falls into place.

Build image once → Bind mount your code → Named volume for deps → nodemon / Vite HMR → Zero rebuild dev loop